
Progetto Mesos — am25

Analisi dei Design Pattern nel Codice Sorgente

Anton Robert, Bortolazzi Filippo, Caliment Sara, Delgrossi Daniele

Il presente documento illustra i principali design pattern architeturali e di progettazione adottati nello sviluppo del gioco da tavolo digitale *Mesos*. Per ciascun pattern vengono descritti il contesto di applicazione, le motivazioni della scelta, la struttura delle classi coinvolte e frammenti di codice estratti direttamente dal progetto.

Pattern individuati	Categorie GoF
11	5

Indice

Panoramica	3
1 Observer	5
1.1 Struttura degli osservatori lato server	5
1.2 Notifica thread-safe con copia difensiva	5
1.3 Variante client — default methods su interfaccia	6
2 Factory Method	7
2.1 Panoramica delle factory	7
2.2 Selezione del prodotto in base al numero di giocatori	7
2.3 Factory client con cache integrata	8
3 Strategy	9
3.1 Gerarchia degli effetti edificio	9
3.2 Gerarchia degli effetti evento	10
3.3 Iniezione della strategia dalla factory	10
4 Memento	11
4.1 Ruoli canonici nel progetto	11
4.2 Interfaccia generica MementoManager	11
4.3 Memento come Java record	12
4.4 Scrittura atomica su disco	12
5 Proxy	14
5.1 Proxy lato server — ClientSocketProxy	14
5.2 Proxy lato client — ServerSocketProxy	14
5.3 Beneficio architetturale	15
6 Command	16
6.1 Interfacce Command	16
6.2 Dispatcher generico	16
6.3 Esempi di Command concreti	16
7 Template Method	18
7.1 Gerarchia Card	18
7.2 Gerarchia BuildingEffect	19
8 Read-only View (Interface Segregation)	20
8.1 BoardView e GameView	20
9 Data Transfer Object (DTO)	22

9.1	Classi DTO presenti	22
9.2	Conversione nel modello — metodo toDTO()	22
10	Flyweight	24
10.1	Cache delle immagini in CardImageFactory	24
11	Functional Interface (variante Command/Strategy)	26
11.1	MarketAction in Game — deduplicazione logica top/bottom	26
11.2	ThrowingRunnable in ServerVirtualView — invio con retry	27
	Considerazioni conclusive	28

Panoramica

Contesto del progetto

Il progetto consiste nello sviluppo di una versione software del gioco da tavolo *Mesos*, realizzata nell'ambito della Prova Finale di Ingegneria del Software del Politecnico di Milano (A.A. 2025–2026). Le specifiche tecniche richiedono l'implementazione di un sistema distribuito con tecnologia **client-server** e impongono esplicitamente l'adozione del pattern architetturale **Model-View-Controller** per la progettazione dell'intero sistema.

Requisiti implementati

I requisiti si suddividono in *game-specific*, relativi alle regole e meccaniche del gioco, e *game-agnostic*, relativi agli aspetti tecnologici e architetturali. In aggiunta ai requisiti obbligatori, il documento ufficiale prevede funzionalità avanzate (FA) facoltative che contribuiscono al punteggio di valutazione.

Si è scelto di implementare il profilo completo di requisiti:

- **Regole complete** — si è optato per l'implementazione integrale delle regole del gioco fisico, incluse le carte personaggio *artista* e *inventore* e i relativi eventi;
- **Interfaccia TUI** — è stata realizzata un'interfaccia testuale a riga di comando, selezionabile all'avvio;
- **Interfaccia GUI** — è stata realizzata un'interfaccia grafica tramite JavaFX, selezionabile in alternativa alla TUI;
- **Protocollo RMI** — la comunicazione client-server può avvenire tramite Java RMI;
- **Protocollo Socket** — la comunicazione può avvenire in alternativa tramite socket TCP/IP; la scelta del protocollo è demandata al client all'avvio;
- **Persistenza (FA)** — è stato scelto di salvare periodicamente lo stato della partita su disco, in modo da consentire il ripristino della sessione anche a seguito di un crash del server;
- **Resilienza alle disconnessioni (FA)** — è stato valutato necessario gestire i casi di disconnessione di rete e di crash del client: i giocatori possono ricollegarsi e la partita prosegue saltando i turni di chi è assente;
- **Classifica su DB (FA)** — si è scelto di mantenere su database MySQL lo storico dei risultati; a fine partita viene mostrata la posizione del giocatore nella classifica globale.

In base alla tabella di valutazione ufficiale, il profilo sopra descritto corrisponde alla fascia massima (**Regole Complete + TUI + GUI + RMI + Socket + 3 FA**).

Struttura del codice

Il codice è organizzato in tre layer principali, coerentemente con il pattern MVC richiesto dalle specifiche:

- `server/model` — modello di dominio: logica di gioco, board, mercato, giocatori, carte ed effetti;
- `server/webLayer`, `client/webLayer` — layer di rete con supporto duale RMI/Socket;
- `client/TUI`, `client/GUI` — layer di presentazione con le due modalità di interfaccia utente.

La notifica degli eventi dal modello verso i client è gestita interamente tramite il pattern Observer (Capitolo 1), che disaccoppia il dominio dal layer di comunicazione. La trasparenza tra i due protocolli di rete è ottenuta tramite il pattern Proxy (Capitolo 5).

La tabella seguente riassume i pattern individuati nell'analisi del codice, la categoria GoF di appartenenza e le classi chiave in cui sono realizzati.

#	Pattern	Categoria	Classe principale
1	Observer	Behavioural — GoF	<code>ServerVirtualView</code>
2	Factory Method	Creational — GoF	<code>DeckFactory</code> , <code>BuildingFactory</code>
3	Strategy	Behavioural — GoF	<code>BuildingEffectInterface</code>
4	Memento	Behavioural — GoF	<code>GameMemento</code> , <code>PersistanceLogger</code>
5	Proxy	Structural — GoF	<code>ClientSocketProxy</code>
6	Command	Behavioural — GoF	<code>ServerToClientMessage</code>
7	Template Method	Behavioural — GoF	<code>Card</code> , <code>BuildingEffect</code>
8	Read-only View (ISP)	Structural — GoF	<code>BoardView</code> , <code>GameView</code>
9	Data Transfer Object (DTO)	Architectural	<code>PlayerDTO</code> , <code>CardDTO</code>
10	Flyweight	Creational — GoF	<code>CardImageFactory</code>
11	Functional Interface	Behavioural — GoF	<code>MarketAction</code> , <code>ThrowingRunnable</code>

1. Observer

Categoria: Behavioural — GoF

Descrizione e motivazione della scelta

Il pattern Observer definisce una dipendenza uno-a-molti tra oggetti: quando un oggetto cambia stato, tutti i suoi dipendenti vengono notificati e aggiornati automaticamente. Si è scelto di fare di questo pattern il meccanismo centrale di notifica dell'intero sistema, in quanto consente di disaccoppiare completamente il modello di dominio dal layer di comunicazione verso i client: il modello non conosce l'esistenza della rete, né il numero di client connessi.

1.1 Struttura degli osservatori lato server

È stato valutato opportuno suddividere il dominio in quattro *Subject* indipendenti, ciascuno con la propria lista di observer e i metodi `addObserver` / `removeObserver`. Le quattro interfacce observer coprono rispettivamente:

- **GameObserver** — cambio di era, fase di gioco, turno giocatore;
- **BoardObserver** — spostamenti dei totem, ordine sulle default tile;
- **MarketObserver** — aggiornamenti del mercato (carte, edifici);
- **PlayerObserver** — variazioni di cibo e punti prestigio.

La classe `ServerVirtualView` implementa tutte e quattro le interfacce contemporaneamente, fungendo da unico punto di raccolta che serializza ogni evento e lo trasmette al client tramite RMI o Socket. Questa scelta concentra in un solo componente tutta la logica di traduzione tra eventi di dominio e messaggi di rete.

1.2 Notifica thread-safe con copia difensiva

Nel metodo privato `notify()` di `Game`, la lista degli observer viene prima copiata con `List.copyOf()` e poi iterata. Si è scelto questo approccio per prevenire `ConcurrentModificationException` nel caso in cui un observer si deregistri durante la notifica, condizione che si verifica tipicamente alla disconnessione di un client.

```
1 private void notify(Consumer<GameObserver> action) {
2     for (GameObserver observer : List.copyOf(observers)) {
3         action.accept(observer);
4     }
5 }
6
7 // Esempio di invocazione:
8 private void notifyGamePhaseChanged() {
9     notify(observer -> observer.onGamePhaseChanged(gamePhase));
10 }
```

Listing 1.1: Notifica thread-safe in `Game.java`

1.3 Variante client — default methods su interfaccia

Sul lato client, si è optato per sfruttare i metodi di default (Java 8) nell'interfaccia `GUIObserver` per rendere ogni callback opzionale. I controller JavaFX eseguono l'override esclusivamente degli eventi di loro pertinenza, eliminando la necessità di classi adapter intermedie e riducendo il boilerplate.

```
1 public interface GUIObserver {  
2     default void onGamePhaseChanged(GAME_PHASE phase)      { }  
3     default void onPlayerAdded(PlayerDTO player)            { }  
4     default void onMarketInitialized(List<CardDTO> top, ...) { }  
5     default void onPlayerDisconnected(String nickname)       { }  
6     // ... 18 callback in totale, tutti con corpo vuoto di default  
7 }
```

Listing 1.2: Interfaccia `GUIObserver.java` con default methods

File e classi di riferimento

```
server/model/Observers/GameObserver.java  
server/model/Observers/BoardObserver.java  
server/model/Observers/MarketObserver.java  
server/model/Observers/PlayerObserver.java  
server/webLayer/ServerVirtualView.java  
client/GUI/GUIObserver.java  
server/model/Game/Game.java
```

2. Factory Method

Categoria: Creational — GoF

Descrizione e motivazione della scelta

Il pattern Factory Method definisce un'interfaccia per la creazione di un oggetto, lasciando ai metodi factory la decisione su quale classe concreta istanziare. Si è scelto di centralizzare la costruzione di tutte le entità di gioco in factory dedicate, in modo che il codice di dominio non abbia mai dipendenze dirette sulle classi concrete, né sulla sorgente dei dati (file JSON, configurazione interna). Nel progetto sono presenti cinque factory lato server e una lato client.

2.1 Panoramica delle factory

Factory	Prodotto	Fonte dati
DeckFactory	Lista di Card per N giocatori	JSON + EventFactory
BuildingFactory	Lista di BuildingCard con effetto legato	building.json
EventFactory	Lista di EventCard con effetto legato	event.json
DefaultTileFactory	Lista di DefaultTile	Configurazione interna
OfferTileFactory	Lista di OfferTile	Configurazione interna
CardImageFactory	ImageView JavaFX con cache	Risorse /images/

2.2 Selezione del prodotto in base al numero di giocatori

È stato necessario adattare la composizione del mazzo e del set di edifici al numero di giocatori (2–5). Si è scelto di incapsulare questa logica direttamente nelle factory: `DeckFactory` seleziona il file JSON corretto tramite uno *switch expression*, mentre `BuildingFactory` utilizza una matrice statica `ERA_COUNTS` per determinare quanti edifici estrarre per ciascuna era.

```
1 String jsonFile = switch (playerNumber) {
2     case 2 -> "/CardResources/json/TwoPlayersCard.json";
3     case 3 -> "/CardResources/json/ThreePlayersCard.json";
4     case 4 -> "/CardResources/json/FourPlayersCard.json";
5     case 5 -> "/CardResources/json/FivePlayersCard.json";
6     default -> { logServerError("Invalid player number"); yield
7         null; }
8 };
```


Listing 2.1: Selezione file JSON in DeckFactory.java

```

1 // Numero di edifici per era, indicizzato per (playerNumber - 2)
2 // Colonne: Era I [0,6), Era II [6,13), Era III [13,21)
3 private static final int[][] ERA_COUNTS = {
4     {1, 2, 3}, // 2 giocatori
5     {2, 2, 4}, // 3 giocatori
6     {2, 3, 4}, // 4 giocatori
7     {2, 3, 5}, // 5 giocatori
8 };

```

Listing 2.2: Matrice ERA_COUNTS in BuildingFactory.java

2.3 Factory client con cache integrata

Sul lato client JavaFX, si è optato per un costruttore privato che rende la classe non istanziabile (tutti metodi statici). La gestione della cache delle immagini è descritta nel Capitolo 10 (Flyweight).

```

1 private CardImageFactory() { } // non istanziabile
2
3 public static ImageView offerTileImageView(char tileID, double
4     fitHeight) {
5     ImageView iv = new ImageView(
6         cached("/images/Tiles/offertiles/" + tileID + "offertile.
7             png")
8     );
9     iv.setFitHeight(fitHeight);
10    iv.setPreserveRatio(true);
11    return iv;
12 }

```

Listing 2.3: Struttura di CardImageFactory.java

File e classi di riferimento

```

server/model/Factory/Deck/DeckFactory.java
server/model/Factory/Building/BuildingFactory.java
server/model/Factory/Event/EventFactory.java
server/model/Factory/DefaultTile/DefaultTileFactory.java
server/model/Factory/OfferTile/OfferTileFactory.java
client/GUI/Controllers/CardImageFactory.java

```

3. Strategy

Categoria: Behavioural — GoF

Descrizione e motivazione della scelta

Il pattern Strategy definisce una famiglia di algoritmi, li incapsula ciascuno in un oggetto separato e li rende intercambiabili. Si è scelto di applicarlo agli effetti delle carte per separare nettamente struttura dati e comportamento: ogni carta delega la propria logica a un oggetto effetto iniettato dalla factory al momento della creazione. In questo modo è possibile aggiungere nuovi effetti senza modificare la gerarchia delle carte, rispettando il principio Open/Closed.

3.1 Gerarchia degli effetti edificio

L'interfaccia `BuildingEffectInterface` definisce il contratto con un unico metodo `applyEffect(Player)`. La classe astratta `BuildingEffect` fornisce il tipo base. Sono stati realizzati 21 implementazioni concrete; alcune sono parametrizzate per coprire più edifici con un'unica classe (ad es. `PPPerCharType` copre 6 edifici distinti variando tipo di carta e moltiplicatore).

```
1  // Interfaccia Strategy
2  public interface BuildingEffectInterface {
3      void applyEffect(Player player);
4  }
5
6  // Classe base astratta
7  public abstract class BuildingEffect implements
8      BuildingEffectInterface {
9      public abstract void applyEffect(Player player);
10 }
11
12 // Strategia parametrizzata (copre 6 edifici diversi)
13 public class PPPerCharType extends BuildingEffect {
14     private final int ppAmount;
15     private final CARD_TYPE targetType;
16
17     public PPPerCharType(int ppAmount, CARD_TYPE targetType) {
18         this.ppAmount = ppAmount;
19         this.targetType = targetType;
20     }
21
22     @Override
23     public void applyEffect(Player player) {
24         int count = player.countCardsOfType(targetType);
25         player.managePP(count * ppAmount);
26     }
27 }
```

```

25     }
26 }

```

Listing 3.1: Contratto Strategy e implementazione parametrizzata

3.2 Gerarchia degli effetti evento

La stessa struttura è stata replicata per gli eventi di gioco. `EventEffectInterface` espone il metodo `solveEvent(List<Player>)`, che agisce sull'intera lista dei giocatori anziché su un singolo. Le implementazioni concrete sono quattro: `HuntEvent`, `SustenanceEvent`, `ShamanEvent` e `ArtistEvent`.

3.3 Iniezione della strategia dalla factory

La `BuildingFactory` inietta la strategia corretta tramite uno *switch* sull'identificativo dell'edificio. Il codice di gioco interagisce esclusivamente con il tipo astratto `BuildingEffect` e non conosce mai la classe concreta dell'effetto.

```

1  private BuildingEffect returnCorrectBuildingEffect(
2      BuildingCard card, BoardView boardView) {
3      return switch (card.getBuildingID()) {
4          case 1  -> new SixFoodCompletedSet();
5          case 2  -> new DiscountFoodOnSustenance(CARD_TYPE.GATHERER)
6              ;
7          case 5  -> {
8              PlusOneFoodOnReturnDefaultTile e =
9                  new PlusOneFoodOnReturnDefaultTile();
10             e.setBoardView(boardView);
11             yield e;
12         }
13         case 14 -> new PPPerCharType(3, CARD_TYPE.HUNTER);
14         case 20 -> new DrawOneMoreCard();
15         case 21 -> new TwentyFivePPEndGame();
16         // ... 21 casi totali
17     };
18 }

```

Listing 3.2: Iniezione della Strategy in `BuildingFactory.java`

File e classi di riferimento

```

server/model/Effect/Building/BuildingEffectInterface.java
server/model/Effect/Building/BuildingEffect.java
server/model/Effect/Event/EventEffectInterface.java
server/model/Effect/Event/EventEffect.java
server/model/Factory/Building/BuildingFactory.java
server/model/Factory/Event/EventFactory.java

```

4. Memento

Categoria: Behavioural — GoF

Descrizione e motivazione della scelta

Il pattern Memento fornisce la capacità di acquisire ed esternalizzare lo stato interno di un oggetto senza violare l'incapsulamento, in modo da poter ripristinare tale stato in un momento successivo. Si è scelto di adottarlo come base per il requisito avanzato di **persistenza**: il server salva periodicamente uno snapshot completo della partita su disco, consentendo il ripristino anche a seguito di un crash.

4.1 Ruoli canonici nel progetto

Ruolo	Classe nel progetto	Responsabilità
Originator	Game, Board, Market, Player	Crea il memento dal proprio stato; si ripristina da esso
Memento	GameMemento, BoardMemento, MarketMemento, PlayerMemento	Snapshot immutabile (Java <code>record</code>)
Caretaker	PersistenceLogger, Controller	Serializza su disco in JSON e ricarica il memento salvato

4.2 Interfaccia generica MementoManager

Si è optato per standardizzare il contratto di salvataggio e ripristino tramite l'interfaccia generica `MementoManager<T>`, implementata da tutti gli Originator. Questo garantisce uniformità e consente di trattare tutti i componenti salvabili in modo omogeneo.

```
1 public interface MementoManager<T> {
2     T createMemento();
3     void restoreMemento(T memento);
4 }
5
6 // Implementazioni:
7 // Game implements MementoManager<GameMemento>
8 // Board implements MementoManager<BoardMemento>
9 // Market implements MementoManager<MarketMemento>
10 // Player implements MementoManager<PlayerMemento>
```

Listing 4.1: `MementoManager.java`

4.3 Memento come Java record

È stato scelto di dichiarare i memento come Java `record`, che garantisce immutabilità per costruzione. Questo li rende snapshot perfetti e thread-safe by design, senza necessità di copie difensive.

```
1 public record GameMemento(  
2     ERA currentEra,  
3     GAME_PHASE gamePhase,  
4     int playerNumber,  
5     String playerToPlaceNickname,  
6     String playerToPlayNickname,  
7     List<PlayerMemento> players,  
8     MarketMemento market,  
9     BoardMemento board  
10 ) { }
```

Listing 4.2: GameMemento.java

4.4 Scrittura atomica su disco

Si è valutato necessario garantire l'integrità del file di salvataggio anche in caso di crash durante la scrittura. La soluzione adottata prevede di scrivere prima su un file temporaneo `.tmp` e poi rinominarlo atomicamente tramite `StandardCopyOption.ATOMIC_MOVE`: se il processo termina a metà scrittura, il file originale rimane integro.

```
1 public void save(GameMemento memento) {  
2     Path tmp = SAVE_PATH.resolveSibling("game_save.tmp");  
3     try {  
4         Files.createDirectories(SAVE_PATH.getParent());  
5         Files.writeString(tmp, new Gson().toJson(memento));  
6         Files.move(tmp, SAVE_PATH,  
7             StandardCopyOption.ATOMIC_MOVE,  
8             StandardCopyOption.REPLACE_EXISTING);  
9     } catch (IOException e) {  
10         logError("Failed to save game state");  
11     }  
12 }
```

Listing 4.3: Salvataggio atomico in PersistenceLogger.java

File e classi di riferimento

```
server/model/persistence/MementoManager.java  
server/model/persistence/GameMemento.java  
server/model/persistence/BoardMemento.java  
server/model/persistence/MarketMemento.java  
server/model/persistence/PlayerMemento.java  
server/model/persistence/PersistenceLogger.java
```

```
server/model/Game/Game.java (createMemento / restoreMemento)
```

5. Proxy

Categoria: Structural — GoF

Descrizione e motivazione della scelta

Il pattern Proxy fornisce un surrogato di un altro oggetto per controllarne l'accesso. Si è scelto di adottarlo per rendere completamente trasparente la differenza tra connessioni RMI e Socket: sia il server che il client operano sempre tramite interfacce comuni, senza alcuna ramificazione condizionale sul protocollo di rete nel codice applicativo. Questo consente di soddisfare il requisito di supporto duale RMI/Socket senza duplicare la logica di gioco.

5.1 Proxy lato server — ClientSocketProxy

Il server utilizza `ClientRemoteInterface` per inviare notifiche ai client. Con RMI il runtime Java genera automaticamente uno stub; con Socket, `ClientSocketProxy` svolge identica funzione serializzando ogni chiamata come un oggetto `ServerToClientMessage` sullo stream di output.

```
1 // Il server invoca sempre la stessa interfaccia,  
2 // indipendentemente dal protocollo del client:  
3 clientStub.gamePhaseChanged(GAME_PHASE.PLACING_PHASE);  
4  
5 // Implementazione del proxy Socket:  
6 public class ClientSocketProxy implements ClientRemoteInterface {  
7     private final ObjectOutputStream out;  
8  
9     @Override  
10    public void gamePhaseChanged(GAME_PHASE phase) throws  
11        RemoteException {  
12        synchronized (out) {  
13            out.writeObject(new GamePhaseChangedMessage(phase));  
14            out.flush();  
15            out.reset();  
16        }  
17    }  
18    // Tutti i metodi di ClientRemoteInterface delegano su out  
19 }
```

Listing 5.1: `ClientSocketProxy.java` — proxy Socket lato server

5.2 Proxy lato client — ServerSocketProxy

Simmetricamente, il client utilizza `ServerRemoteInterface` per inviare comandi al server. Con RMI il lookup restituisce uno stub; con Socket, `ServerSocketProxy` serializza ogni chiamata come un `ClientToServerMessage`.

```
1 public class ServerSocketProxy implements ServerRemoteInterface {
2
3     private void send(ClientToServerMessage msg) throws
4         RemoteException {
5         try {
6             synchronized (out) {
7                 out.writeObject(msg);
8                 out.flush();
9                 out.reset();
10            }
11        } catch (IOException e) {
12            throw new RemoteException("Socket write error", e);
13        }
14
15        @Override
16        public void placePlayer(int position) throws RemoteException {
17            send(new PlacingPlayerMessage(position));
18        }
19    }
```

Listing 5.2: ServerSocketProxy.java — proxy Socket lato client

5.3 Beneficio architetturale

Grazie ai due proxy, l'intero codice applicativo (Controller, TUI, GUI) è scritto una sola volta contro le interfacce di rete. Sostituire il protocollo Socket con un'alternativa richiederebbe la sola riscrittura delle due classi proxy, senza modificare il dominio né le viste.

File e classi di riferimento

```
server/webLayer/Socket/ClientSocketProxy.java
client/webLayer/Socket/ServerSocketProxy.java
server/webLayer/RMI/ClientRemoteInterface.java
client/webLayer/RMI/ServerRemoteInterface.java
```


6. Command

Categoria: Behavioural — GoF

Descrizione e motivazione della scelta

Il pattern Command incapsula una richiesta come oggetto autonomo, permettendo di parametrizzare, serializzare e dispatchare operazioni in modo uniforme. Si è optato per questo pattern come base del protocollo di comunicazione Socket: ogni messaggio di rete è un oggetto Command serializzabile che porta in sé i dati e il metodo da invocare. Questo consente al dispatcher di essere completamente generico e di aggiungere nuovi tipi di messaggio senza modificarne il codice.

6.1 Interfacce Command

Sono state definite due famiglie di Command, una per ciascuna direzione di comunicazione.

```
1 // Server -> Client
2 public interface ServerToClientMessage extends Serializable {
3     void execute(ClientRemoteInterface client) throws Exception;
4 }
5
6 // Client -> Server
7 public interface ClientToServerMessage extends Serializable {
8     void execute(ServerRemoteInterface server,
9                 ClientRemoteInterface proxy) throws Exception;
10 }
```

Listing 6.1: Interfacce Command del protocollo Socket

6.2 Dispatcher generico

Il dispatcher lato server (`SocketClientHandler`) non conosce il tipo concreto del messaggio: deserializza l'oggetto e invoca `execute()`. L'aggiunta di un nuovo tipo di messaggio non richiede alcuna modifica al codice del dispatcher.

```
1 // Loop di ricezione --- dispatch completamente polimorfico
2 while (true) {
3     ClientToServerMessage message =
4         (ClientToServerMessage) in.readObject();
5     message.execute(serverLogic, clientSocketProxy);
6 }
```

Listing 6.2: Dispatcher in `SocketClientHandler.java`

6.3 Esempi di Command concreti

```
1 // Messaggio Server -> Client: notifica cambio fase
```

```
2 public class GamePhaseChangedMessage implements
    ServerToClientMessage {
3     private final GAME_PHASE phase;
4
5     public GamePhaseChangedMessage(GAME_PHASE phase) { this.phase =
        phase; }
6
7     @Override
8     public void execute(ClientRemoteInterface client) throws
        Exception {
9         client.gamePhaseChanged(phase);
10    }
11 }
12
13 // Messaggio Client -> Server: piazzamento totem
14 public class PlacingPlayerMessage implements ClientToServerMessage
    {
15     private final int position;
16
17     @Override
18     public void execute(ServerRemoteInterface server,
        ClientRemoteInterface proxy) throws
19         Exception {
20         server.placePlayer(position);
21     }
22 }
```

Listing 6.3: Command concreti del protocollo

File e classi di riferimento

server/webLayer/Socket/ServerToClientMessage.java
client/webLayer/Socket/ClientToServerMessage.java
server/webLayer/Socket/messages/gameMessages/GamePhaseChangedMessage.java
client/webLayer/Socket/messages/PlacingPlayerMessage.java
server/webLayer/Socket/SocketClientHandler.java
client/webLayer/Socket/ServerListener.java

7. Template Method

Categoria: Behavioural — GoF

Descrizione e motivazione della scelta

Il pattern Template Method definisce lo scheletro di un algoritmo all'interno di un metodo, delegando alcuni passi alle sottoclassi. Si è scelto di adottarlo per definire il contratto obbligatorio di ogni carta del gioco e di ogni effetto edificio: le classi astratte impongono i metodi che ogni specializzazione deve implementare, garantendo che la logica di mercato e di risoluzione degli effetti possa operare polimorficamente sul tipo base senza conoscere i dettagli concreti.

7.1 Gerarchia Card

La classe astratta **Card** obbliga ogni sottoclasse a implementare tre operazioni fondamentali. L'intera logica di mercato opera sul tipo astratto **Card** e invoca polimorficamente questi metodi.

```
1 public abstract class Card {
2     protected ERA era;
3     protected CARD_TYPE cardType;
4
5     // Passo 1 --- comportamento specifico all'aggiunta in tribu'
6     public abstract void addCardToPlayer(Player player)
7         throws NotSelectableCardException;
8
9     // Passo 2 --- serializzazione verso il client
10    public abstract CardDTO toDTO();
11
12    // Passo 3 --- confronto durante il ripristino da memento
13    public abstract boolean matchesDTO(CardDTO dto);
14 }
15
16 // Specializzazione concreta:
17 public class HuntersCard extends Card {
18     private final boolean hasIcon;
19
20     @Override
21     public void addCardToPlayer(Player player) { player.addHunter(
22         hasIcon); }
23
24     @Override
25     public CardDTO toDTO() { return new CardDTO(era, cardType,
26         hasIcon); }
```

```
26     @Override
27     public boolean matchesDTO(CardDTO dto) {
28         return dto.getCardType() == CARD_TYPE.HUNTER
29             && dto.isHasIcon() == hasIcon;
30     }
31 }
```

Listing 7.1: Classe astratta Card.java

7.2 Gerarchia BuildingEffect

La classe BuildingEffect svolge un ruolo analogo per gli effetti degli edifici: impone un'unica firma `applyEffect(Player)` su cui il codice di gioco opera polimorficamente.

```
1 public abstract class BuildingEffect implements
    BuildingEffectInterface {
2     public abstract void applyEffect(Player player);
3 }
4
5 // Specializzazione:
6 public class BuilderDoublePP extends BuildingEffect {
7     @Override
8     public void applyEffect(Player player) {
9         int builders = player.countCardsOfType(CARD_TYPE.BUILDER);
10        player.managePP(builders * 2);
11    }
12 }
```

Listing 7.2: BuildingEffect.java e specializzazione

File e classi di riferimento

```
server/model/Card/Card.java
server/model/Card/HuntersCard.java
server/model/Card/ArtistCard.java
server/model/Card/BuildingCard.java
server/model/Effect/Building/BuildingEffect.java
```

8. Read-only View (Interface Segregation)

Categoria: Structural — GoF

Descrizione e motivazione della scelta

Pur non essendo un pattern GoF con nome canonico, la Read-only View è una scelta progettuale che applica il principio ISP (Interface Segregation Principle). Si è valutato opportuno esporre ai componenti che necessitano solo di leggere lo stato del modello un'interfaccia ristretta, impedendo modifiche accidentali dall'interno di effetti o factory e rendendo esplicite le dipendenze di sola lettura.

8.1 BoardView e GameView

Board implementa sia BoardView (lettura) sia MementoManager<BoardMemento> (salvataggio). Si è scelto di passare esclusivamente BoardView alla BuildingFactory e agli effetti edificio che necessitano di consultare la posizione dei giocatori, applicando il principio del minimo privilegio.

```
1 // BoardView: espone solo la lettura del board
2 public interface BoardView {
3     List<Player> getOrderedPlayerOnOfferTile();
4     List<Player> getOrderedPlayerOnDefaultTile();
5     OfferTile getCopyTilePlayerIsOn(Player player);
6     boolean isPlayerOnAnEligibleDefaultTile(Player player);
7 }
8 // I metodi mutanti (placePlayer, returnOnDefaultTiles...) non sono
   // esposti.
9
10 // GameView: espone solo la lettura del gioco
11 public interface GameView {
12     int getPlayerNumber();
13     List<Player> getPlayerList();
14     ERA getCurrentEra();
15     void nextEra();
16 }
```

Listing 8.1: Interfacce read-only del modello

Ad esempio, PlusOneFoodOnReturnDefaultTile può interrogare la posizione di un giocatore tramite BoardView senza avere accesso ai metodi di modifica del board.

File e classi di riferimento

server/model/Board/BoardView.java
server/model/Game/GameView.java

```
server/model/Effect/Building/PlusOneFoodOnReturnDefaultTile.java  
server/model/Factory/Building/BuildingFactory.java
```

9. Data Transfer Object (DTO)

Categoria: Architectural

Descrizione e motivazione della scelta

Il pattern DTO separa il modello di dominio dal layer di trasporto. Si è scelto di introdurre una controparte DTO per ogni entità principale del modello: oggetti serializzabili, privi di logica di business, usati come unica valuta di scambio tra server e client. Questa separazione permette di modificare il modello senza impattare il protocollo di rete e viceversa.

9.1 Classi DTO presenti

- **PlayerDTO** — nickname, quantità di cibo, punti prestigio, colore totem;
- **CardDTO** — tipo di carta, era, attributi specifici per tipo;
- **BuildingDTO** — identificativo edificio, costo in cibo, PP fine partita;
- **OffertileDTO** / **DefaultTileDTO** — stato corrente delle tessere;
- **MarketDTO** — snapshot completo del mercato;
- **EventDTO** — tipo e identificativo dell'evento;
- **ActionDTO** — azioni di pesca disponibili per il giocatore corrente.

9.2 Conversione nel modello — metodo `toDTO()`

Si è scelto di attribuire al modello la responsabilità della propria serializzazione in DTO, esponendo un metodo `toDTO()` su ogni entità. La **ServerVirtualView** aggiorna i propri snapshot locali ad ogni callback observer e li usa per il resync dei client che si riconnettono.

```
1  @Override
2  public void onMarketChanged(List<Card> topCards,
3                               List<Card> bottomCards, ...) {
4      this.topCards = new ArrayList<>(
5          topCards.stream().map(Card::toDTO).toList()
6      );
7      this.topBuildings = new ArrayList<>(
8          topBuildings.stream()
9              .map(b -> (BuildingDTO) b.toDTO())
10             .toList()
11      );
12      submitTask(() -> clientStub.initializeMarket(
13          this.topCards, this.bottomCards,
14          this.topBuildings, this.bottomBuildings
15      ));
16 }
```

Listing 9.1: Aggiornamento snapshot DTO in `ServerVirtualView.java`

File e classi di riferimento

server/webLayer/DTOs/PlayerDTO.java
server/webLayer/DTOs/CardDTO.java
server/webLayer/DTOs/BuildingDTO.java
server/webLayer/DTOs/MarketDTO.java
server/webLayer/ServerVirtualView.java

10. Flyweight

Categoria: Creational — GoF

Descrizione e motivazione della scelta

Il pattern Flyweight riduce il costo in termini di memoria e I/O condividendo lo stato tra oggetti che vengono richiesti ripetutamente. Si è valutato che nell'interfaccia JavaFX le stesse immagini vengono caricate più volte da componenti distinti: per evitare I/O ridondante e ridurre il consumo di memoria, si è scelto di introdurre una cache statica delle risorse grafiche.

10.1 Cache delle immagini in CardImageFactory

La classe `CardImageFactory` mantiene una `ConcurrentHashMap` statica che mappa il percorso della risorsa all'istanza `Image` di JavaFX corrispondente. Ogni immagine viene caricata dal classpath una sola volta; le richieste successive restituiscono l'istanza già in cache.

```
1 private static final Map<String, Image> IMAGE_CACHE =
2     new ConcurrentHashMap<>();
3
4 private static Image cached(String path) {
5     return IMAGE_CACHE.computeIfAbsent(path,
6         p -> new Image(
7             Objects.requireNonNull(
8                 CardImageFactory.class.getResourceAsStream(p)
9             )
10        );
11 };
12 }
13
14 // Ogni metodo pubblico delega a cached():
15 public static ImageView offerTileImageView(char tileID, double
16     fitHeight) {
17     ImageView iv = new ImageView(
18         cached("/images/Tiles/offertiles/" + tileID + "offertile.
19             png")
20     );
21     iv.setFitHeight(fitHeight);
22     iv.setPreserveRatio(true);
23     return iv;
24 }
```

Listing 10.1: Cache Flyweight in `CardImageFactory.java`

Si è optato per `ConcurrentHashMap` e `computeIfAbsent` per garantire correttezza in contesti multi-thread, rilevante poiché il rendering JavaFX può coinvolgere thread distinti.

File e classi di riferimento

client/GUI/Controllers/CardImageFactory.java

11. Functional Interface (variante Command/Strategy)

Categoria: Behavioural — GoF

Descrizione e motivazione della scelta

Le interfacce funzionali introdotte in Java 8 offrono una forma idiomatica e sintetica di implementare i pattern Command e Strategy tramite espressioni lambda. Si è scelto di ricorrere a questo approccio in due punti del codice dove la duplicazione sarebbe altrimenti inevitabile, rendendo espliciti i comportamenti variabili attraverso parametri funzionali.

11.1 MarketAction in Game — deduplicazione logica top/bottom

La logica di selezione di una carta dal mercato è identica per la lista superiore e quella inferiore, a eccezione del metodo da invocare e del contatore da decrementare. Anziché duplicare il codice, si è scelto di parametrizzare questi comportamenti tramite l'interfaccia funzionale privata MarketAction.

```
1 @FunctionalInterface
2 private interface MarketAction {
3     void execute(Player player, int position)
4         throws IndexOutOfBoundsException, NotEnoughFoodException,
5             NotSelectableCardException, EmptyMarketException;
6 }
7
8 // Unica implementazione dell'algoritmo condiviso:
9 private void selectGenericCard(
10     CARD_TYPE type, int position, Player player,
11     MarketAction buyBuilding, MarketAction selectCard,
12     String rowLabel, Runnable subtractAction) throws ... {
13     switch (type) {
14         case BUILDING -> buyBuilding.execute(player, position);
15         case EVENT     -> throw new NotSelectableCardException(...);
16         default        -> selectCard.execute(player, position);
17     }
18     subtractAction.run();
19     notifyActionChanged();
20     if (noMoreActionsLeft()) throw new NoMoreActionToDo();
21 }
22
23 // I due chiamanti differiscono solo nelle lambda passate:
24 public void selectGenericCardTopLists(...) {
25     selectGenericCard(type, pos, player,
26         (p, i) -> market.buyBuildingTopList(i, p),
27         (p, i) -> market.selectCardFromTopList(i, p),
28         "top list",
```

```

29         () -> offertile.getActionAvailable().subtractOneTopAction()
30     };
31 }

```

Listing 11.1: MarketAction in Game.java

11.2 ThrowingRunnable in ServerVirtualView — invio con retry

Si è valutato necessario gestire in modo uniforme i fallimenti RMI su ogni notifica verso il client, con retry automatico. L'interfaccia funzionale privata `ThrowingRunnable` consente di passare all'executor qualsiasi lambda che possa lanciare `RemoteException`, centralizzando la logica di retry in un unico metodo `submitTask()`.

```

1  @FunctionalInterface
2  private interface ThrowingRunnable {
3      void run() throws RemoteException;
4  }
5
6  private void submitTask(ThrowingRunnable task) {
7      if (!connected) return;
8      executor.submit(() -> {
9          int attempt = 0;
10         while (true) {
11             try {
12                 task.run();
13                 return;
14             } catch (RemoteException e) {
15                 if (++attempt > MAX_RETRIES) {
16                     disconnectCallback.run();
17                     return;
18                 }
19                 Thread.sleep(RETRY_DELAY_MS);
20             }
21         }
22     });
23 }
24
25 // Utilizzo uniforme per ogni notifica al client:
26 submitTask(() -> clientStub.gamePhaseChanged(currentGamePhase));
27 submitTask(() -> clientStub.playerUpdatePP(nickname, newPP));

```

Listing 11.2: ThrowingRunnable in ServerVirtualView.java

File e classi di riferimento

server/model/Game/Game.java (MarketAction, selectGenericCard)

server/webLayer/ServerVirtualView.java (ThrowingRunnable, submitTask)

Considerazioni conclusive

I design pattern adottati non sono stati applicati in modo meccanico, ma sono stati scelti per rispondere a specifiche esigenze architetturali emerse durante lo sviluppo. Di seguito si riassumono le principali considerazioni che hanno guidato le scelte progettuali.

Aspetto	Considerazione
Observer come backbone del sistema di notifica	Si è scelto di costruire l'intera pipeline Model-View sull'Observer. La suddivisione in quattro interfacce distinte rispetta l'ISP e consente a <code>ServerVirtualView</code> di ricevere tutti gli eventi senza accoppiamenti superflui. I default methods su <code>GUIObserver</code> eliminano la necessità di classi adapter lato client.
Factory + Strategy per un sistema di carte estensibile	La combinazione Factory + Strategy rende il sistema delle carte aperto all'estensione: aggiungere un nuovo edificio richiede una nuova classe effetto e una riga nello switch della factory, senza modificare il codice di gioco esistente (principio Open/Closed).
Proxy per la trasparenza di protocollo	Si è optato per il Proxy per eliminare qualsiasi dipendenza dal protocollo nel codice applicativo. Tutta la logica di gioco è scritta una sola volta contro le interfacce; cambiare protocollo richiederebbe la sola sostituzione dei due proxy.
Memento per la persistenza robusta	L'adozione di Java <code>record</code> come memento e la scrittura atomica su disco garantiscono robustezza in caso di crash. È stato valutato che questa soluzione soddisfa interamente il requisito avanzato di persistenza senza violare l'incapsulamento del modello.
Command per l'estensibilità del protocollo	Ogni nuovo tipo di messaggio di rete è una classe Command autonoma. Il dispatcher rimane invariato all'aggiunta di nuove funzionalità, il che ha consentito di integrare heartbeat, extra draw e classifica in modo incrementale durante lo sviluppo.